

Kubernetes for QA

Wprowadzenie do konteneryzacji i Docker

```
docker run nginx
```



Agenda

Wirtualizacja

Konteneryzacja

Docker jako przykład

Krótkie demo Docker

Dlaczego potrzebujemy Kubernetes?



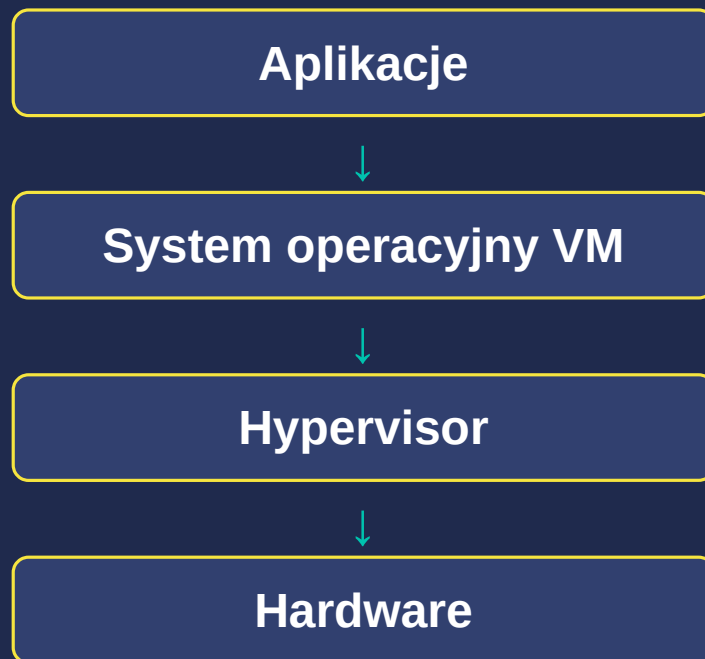
Problem przed kontenerami

1 aplikacja = 1 serwer

niskie wykorzystanie zasobów
trudne utrzymanie i aktualizacje
problemy z zależnościami aplikacji
różnice między środowiskami



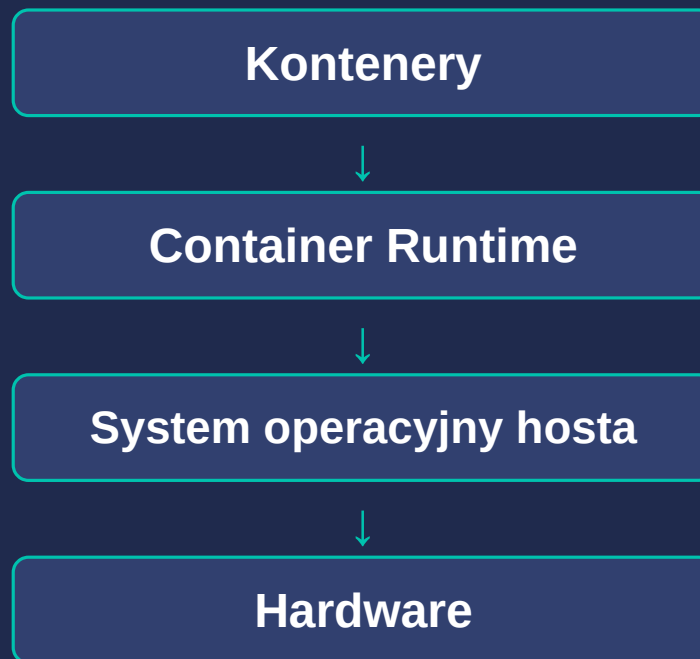
Wirtualizacja



plus: izolacja minus: każdy system ma własny OS



Konteneryzacja



szybki start mniejsze zużycie zasobów przenośność



VM vs Container

VM

- własny OS
- większe zużycie RAM/CPU
- wolniejszy start
- mocna izolacja

Container

- współdzielony kernel
- lekki
- start w sekundach
- łatwy do przenoszenia



Czym jest Docker?

Docker to narzędzie do pracy z kontenerami

- budowanie obrazów
- uruchamianie kontenerów
- zarządzanie lokalnym środowiskiem
- dystrybucja aplikacji przez registry



Image → Container



Obraz jest szablonem. Kontener jest uruchomioną instancją obrazu.



Demo: uruchomienie kontenera

```
docker run -d --name web nginx
```

Efekt: lokalny kontener z serwerem nginx działa w tle.



Demo: lista kontenerów

```
docker ps
```

Pokazuje aktualnie działające kontenery.



Demo: logi kontenera

```
docker logs web
```

To samo podejście później zobaczymy w Kubernetes: `kubectl logs`.



Demo: wejście do kontenera

```
docker exec -it web bash  
# albo: docker exec -it web sh
```

To samo podejście później zobaczymy w Kubernetes: `kubectl exec`.



Problem skali

Docker uruchamia kontenery. Ale kto nimi zarządza?

10 kontenerów?

100 kontenerów?

1000 kontenerów?



Potrzeba orkiestracji

gdzie uruchomić kontener?
co jeśli kontener się wyłączy?
jak skalować aplikację?
jak bezpiecznie aktualizować wersję?

To jest moment, w którym pojawia się Kubernetes.



Kubernetes

Docker uruchamia kontenery.

Kubernetes zarządza kontenerami.

Następny krok: architektura Kubernetes i pierwsza aplikacja.

